OrderBook C++

Aakash Tarang

May 2026

1 Introduction

This document is meant to be self notes while learning to make low latency systems, I have chosen to implement a Order Book using C++ and will be tracking my progress.

2 First Attempt

I completed my first attempt with a very basic implementation of a limit order book along with a simple client-server setup for latency testing. The goal of this implementation was not absolute performance optimization, but rather to establish a working baseline architecture capable of:

- Receiving orders over a TCP connection
- Parsing and inserting orders into the order book
- Matching compatible bid and ask orders
- Sending acknowledgements back to the client
- Measuring end-to-end latency

The implementation used standard C++ data structures and a synchronous networking model.

2.1 Setting up the communication

The communication layer was implemented using TCP sockets over the loopback interface (`127.0.0.1`). A dedicated server process maintained the order book while a separate client process continuously generated random orders and measured round-trip latency.

The server socket setup was implemented as follows:

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

sockaddr_in server_addr{};
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY;

```

```
bind(server_fd, (sockaddr*)&server_addr, sizeof(server_addr));
listen(server_fd, 5);

int client_socket = accept(server_fd, nullptr, nullptr);
```

Listing 1: Server Socket Setup

The client connected to the server using the loopback address:

```
int sock_fd = socket(AF_INET, SOCK_STREAM, 0);

sockaddr_in server_addr{};
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);

inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

connect(sock_fd, (sockaddr*)&server_addr,
        sizeof(server_addr));
```

Listing 2: Client Connection Setup

Orders were transmitted as newline-delimited text messages of the form:

```
BID 101 5
```

or

```
ASK 99 2
```

Since TCP is a byte-stream protocol and does not preserve message boundaries, the server maintained an intermediate buffer and parsed complete messages only after encountering a newline character.

```
leftover += string(buffer, bytes);

while ((pos = leftover.find('\n')) != string::npos) {
    string message = leftover.substr(0, pos);
    leftover = leftover.substr(pos + 1);

    stringstream ss(message);

    string side_str;
    int price, qty;

    ss >> side_str >> price >> qty;
}
```

Listing 3: Message Parsing Logic

This ensured that partial TCP reads or multiple concatenated messages could still be handled correctly.

2.2 Order Book Implementation

The order book itself was implemented using two priority queues:

- A max-heap for bids
- A min-heap for asks

```
priority_queue<Order,
               vector<Order>,
               CompareBid> bids;

priority_queue<Order,
               vector<Order>,
               CompareAsk> asks;
```

Listing 4: Order Book Data Structures

Price-time priority was implemented using custom comparators. Higher bid prices received priority while lower ask prices were prioritized on the sell side. For equal prices, lower order IDs were prioritized to preserve FIFO ordering.

Matching was performed whenever the highest bid price crossed the lowest ask price.

```
while (!bids.empty() &&
       !asks.empty() &&
       bids.top().price >= asks.top().price) {

    Order bid = bids.top();
    Order ask = asks.top();

    bids.pop();
    asks.pop();

    int traded_qty = min(bid.qty, ask.qty);

    bid.qty -= traded_qty;
    ask.qty -= traded_qty;

    if (bid.qty > 0)
        bids.push(bid);

    if (ask.qty > 0)
        asks.push(ask);
}
```

Listing 5: Matching Logic

2.3 Latency Benchmarking

The client continuously generated random orders and measured round-trip latency between sending an order and receiving the acknowledgement from the server.

```
auto t1 =
std::chrono::high_resolution_clock::now();

send(sock_fd, msg.c_str(), msg.size(), 0);

uint8_t ack = 0;
read(sock_fd, &ack, 1);

auto t2 =
std::chrono::high_resolution_clock::now();

long long latency =
std::chrono::duration_cast<
std::chrono::microseconds>(t2 - t1).count();
```

Listing 6: Latency Measurement

A total of 1000 orders were transmitted during the benchmark.
The observed results were:

Metric	Latency
Minimum	20 μs
Maximum	162 μs
Mean	47 μs

These numbers already demonstrated that even a naive synchronous implementation on loopback can achieve sub-100 microsecond average latency.

However, the variance between the best and worst cases indicated significant scheduling and operating system interference.

2.4 Jitter Analysis

The benchmark recorded a minimum latency of 20 μs, representing a clean run with no OS interruptions, a maximum of 162 μs where the scheduler paused the process mid-measurement, and a mean of 47 μs for a typical round trip on loopback. The gap between min and max is the critical observation — jitter of 142 μs relative to a 47 μs mean indicates the OS is interfering significantly and that the mean itself is an unreliable performance indicator.

The latency over time plot reveals three distinct phases. In the first 330 iterations latency runs high at 60–147 μs due to cold caches, TCP slow start, and heap allocator warmup — all data structures are being accessed for the first time and every access is a cache miss. At iteration 330 there is a sharp cliff down to roughly 35 μs as the system hits steady state: frequently accessed data is warm in cache, the allocator has a pool of reusable blocks, and the OS scheduler has learned the process pattern. From iteration 330 onward latency stabilizes in a 25–45 μs band with occasional spikes caused by context switches and cache evictions from background OS activity. This warmup effect means the reported mean of 47 μs is inflated — the true steady state mean is closer to 35 μs, and benchmarks should discard the first 300 iterations before recording samples.

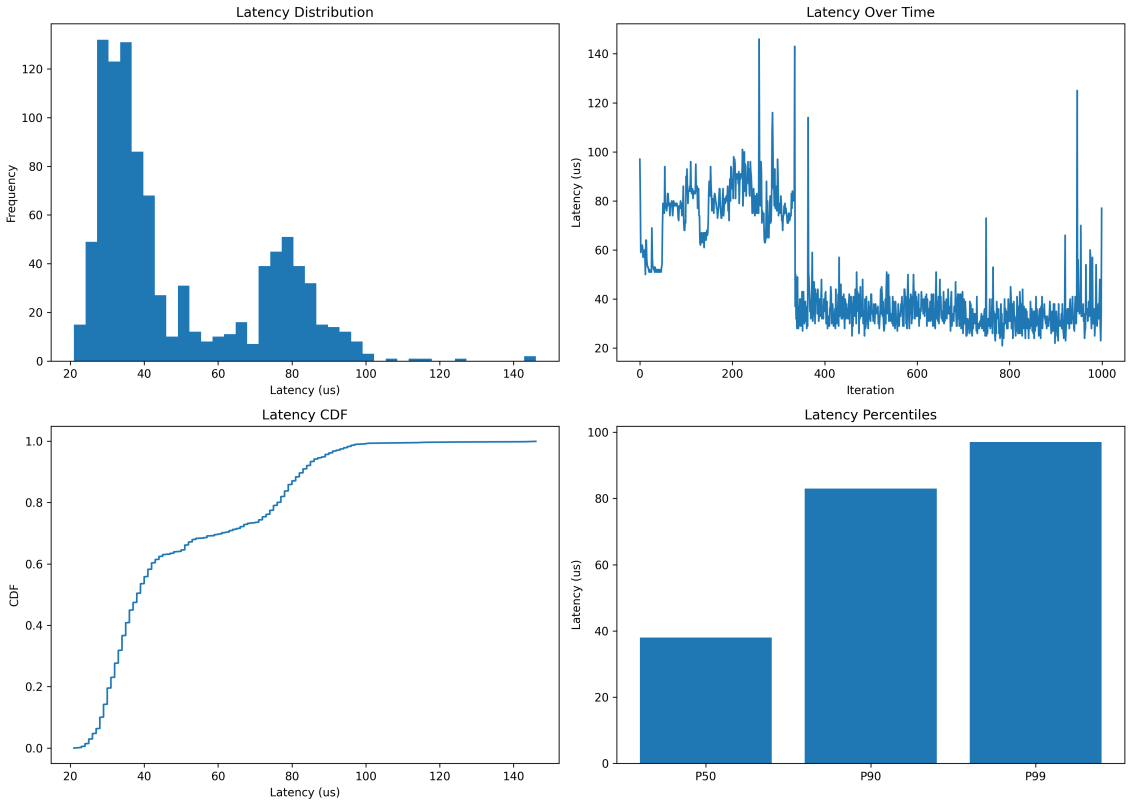


Figure 1: Latency analysis

2.5 Flaws in the first implementation

- `priority_queue` stores orders scattered across the heap. Every `top()` and `pop()` potentially pulls a different cache line from RAM. This introduces cache misses and increases latency.
- String-based message protocol forces heap allocation on every order. Operations like `std::string`, `stringstream`, and string concatenation allocate and free memory dynamically, causing unpredictable latency spikes in the hot path.
- No framing protocol on the TCP stream. TCP delivers a byte stream with no message boundaries. A single `read()` call may return multiple concatenated orders or a partial order. Without a length-prefix or complete read loop, the parser silently corrupts or drops orders under load.
- Single-threaded blocking design. The server blocks on `accept()` waiting for one client, then blocks on `read()` waiting for each order. There is no concurrency — a slow client stalls the entire matching engine.
- No error checking on socket operations. `send()` and `read()` can return partial results or

fail silently. The implementation assumes every call succeeds and transfers the full message, which is not guaranteed by the OS.

- `cout` inside the matching hot path. Writing to stdout on every order and every trade is a system call that blocks until the terminal flushes. This artificially inflates latency measurements and would be unacceptable in production.
- Order book state is unbounded in memory. Unmatched orders accumulate in the priority queues indefinitely with no cancellation, expiry, or memory reclamation mechanism. Under sustained load the process grows without bound.
- No order cancellation or modification. A real order book must support cancel and replace operations. Without them the book fills with stale orders that can never be matched or removed.

2.6 Measurement Improvement

The initial benchmark implementation introduced significant overhead due to repeated terminal output inside the critical request-response path. Both the client and server were performing `cout` operations for every processed order, which resulted in additional system calls, buffering overhead, synchronization costs, and scheduler interruptions. As a consequence, the measured latency reflected not only the networking and matching engine performance, but also terminal I/O overhead.

To obtain a more accurate estimate of the order processing latency, all logging operations were removed from the hot execution path. After this optimization, the benchmark was executed for 100000 orders over a localhost TCP connection on an Apple M1 system.

Metric	Latency
Minimum	10 μs
Maximum	231 μs
Mean	15 μs
P50	15 μs
P90	17 μs
P99	24 μs

The reduction in both average latency and tail latency demonstrates the substantial impact of unnecessary I/O operations on low-latency systems. Although this benchmark is not a perfectly isolated measurement of the matching engine itself, it serves as a useful end-to-end latency benchmark for evaluating future optimizations to the order book implementation. In particular, it provides a practical baseline for studying the effects of protocol design, parsing overhead, memory allocation, and networking optimizations on overall system latency.

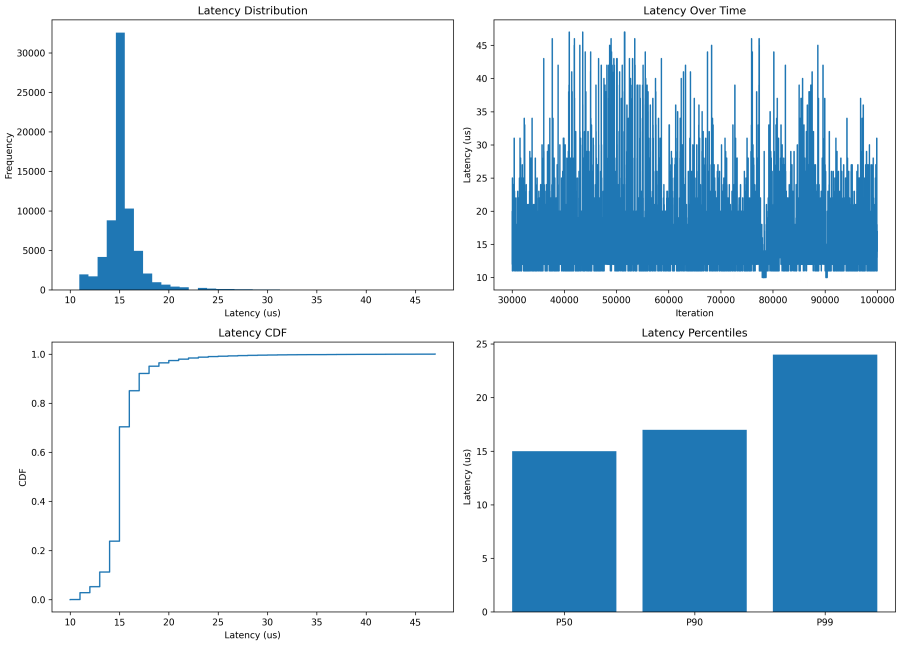


Figure 2: Latency analysis after removing warm-up iterations and extreme outliers

3 Second Attempt

3.1 Order Book Implementation

This time the order book was implemented using a price-indexed vector of price levels, improving cache performance over the prior `priority_queue`-based design, which suffered from the following limitation:

- `priority_queue` stores orders scattered across the heap. Every `top()` and `pop()` potentially pulls a different cache line from RAM. This introduces cache misses and increases latency.

Instead, the order book is backed by two flat vectors of `PriceLevel` structs, one for bids and one for asks, indexed directly by price tick.

```
struct PriceLevel {
    queue<uint64_t> orders; // stores order IDs in FIFO order
    int total_qty = 0;
};

struct OrderBook {
    static const int MIN_PRICE = 0;
    static const int MAX_PRICE = 10000;
    static const int TICK_SIZE = 5;

    vector<PriceLevel> asks;
```

```
    vector<PriceLevel> bids;
    unordered_map<uint64_t, Order> order_map;

    int best_bid = INT_MIN;
    int best_ask = INT_MAX;
};
```

Listing 7: Order Book Data Structures

Each price level maps directly to a vector index via a tick conversion, giving O(1) access to any price level without heap traversal.

```
int to_index(int raw_price) {
    if (raw_price > MAX_PRICE || raw_price < MIN_PRICE) return -1;
    return (raw_price - MIN_PRICE) / TICK_SIZE;
}
```

Listing 8: Price to Index Conversion

Price-time priority is preserved naturally: each `PriceLevel` holds a `queue` of order IDs, so orders at the same price are matched strictly in FIFO order. The best bid and best ask prices are tracked explicitly as integers, updated incrementally on each add, cancel, and match, avoiding any scan in the common case. When a level is exhausted, a linear scan finds the next occupied level.

```
void match() {
    while (best_bid >= best_ask) {
        PriceLevel& bid_level = bids[to_index(best_bid)];
        PriceLevel& ask_level = asks[to_index(best_ask)];

        // skip cancelled orders at front of each level
        while (!bid_level.orders.empty()) {
            auto it = order_map.find(bid_level.orders.front());
            if (it != order_map.end() && !it->second.cancelled) break;
            if (it != order_map.end()) order_map.erase(it);
            bid_level.orders.pop();
        }
        while (!ask_level.orders.empty()) {
            auto it = order_map.find(ask_level.orders.front());
            if (it != order_map.end() && !it->second.cancelled) break;
            if (it != order_map.end()) order_map.erase(it);
            ask_level.orders.pop();
        }

        if (bid_level.orders.empty()) {
            best_bid = find_next_bid(to_index(best_bid) - 1); break;
        }
        if (ask_level.orders.empty()) {
            best_ask = find_next_ask(to_index(best_ask) + 1); break;
        }

        Order& bid = order_map[bid_level.orders.front()];
        Order& ask = order_map[ask_level.orders.front()];
```

```
29
30          int traded_qty = min(bid.qty, ask.qty);
31          bid.qty -= traded_qty;
32          ask.qty -= traded_qty;
33          bid_level.total_qty -= traded_qty;
34          ask_level.total_qty -= traded_qty;
35
36          if (bid.qty == 0) {
37              order_map.erase(bid.id); bid_level.orders.pop();
38          }
39          if (ask.qty == 0) {
40              order_map.erase(ask.id); ask_level.orders.pop();
41          }
42
43          if (bid_level.orders.empty())
44              best_bid = find_next_bid(to_index(best_bid) - 1);
45          if (ask_level.orders.empty())
46              best_ask = find_next_ask(to_index(best_ask) + 1);
47      }
48  }
```

Listing 9: Matching Logic

Since both `bids` and `asks` are contiguous vectors, accessing nearby price levels reuses the same cache lines, eliminating the pointer-chasing overhead of heap-based traversal. Order lookup and cancellation are handled via an `unordered_map` keyed on order ID, providing O(1) average-case access. Lazy cancellation is used: cancelled orders are marked in the map and skipped at match time, avoiding the O(n) cost of removing an arbitrary element from the middle of a heap.

3.2 Measurement Bottleneck and Direct Benchmarking

The initial benchmarking strategy was designed to simulate a realistic deployment environment, where orders were transmitted over a TCP socket and latency was measured as the full round-trip time between transmission and acknowledgement. However, this methodology produced nearly identical results for both implementations, with average latencies of approximately $15\,\mu s$ in each case.

This similarity was not due to architectural equivalence between the two order book designs. Instead, the dominant cost originated from the networking stack itself. The loopback TCP round-trip — including kernel context switches, socket buffer management, and the `read`/`write` system calls — contributed roughly 14–$15\,\mu s$ per operation. As a result, the actual order book processing time, which occurred entirely in userspace, accounted for less than 5% of the measured latency. Consequently, any performance difference between the two data structures became indistinguishable at microsecond resolution.

To isolate the performance of the matching engines themselves, both implementations were subsequently benchmarked entirely in-process, eliminating all networking and IPC overhead. Orders were pre-generated before entering the timed region in order to exclude random number generation costs. Each individual `add_order()` invocation was then measured independently using `chrono::high_resolution_clock` at nanosecond resolution. The resulting measurements are

Metric	Priority Queue	Vector + Hash Map
Mean (filtered)	212.9 ns	174.1 ns
P50	83 ns	84 ns
P90	542 ns	375 ns
P99	917 ns	791 ns

Table 1: Per-order in-process latency measurements after outlier filtering

The vector-based implementation demonstrates an approximately **18% reduction in mean latency** and a **31% reduction at the P90 percentile** relative to the priority queue implementation. The P50 values remain nearly identical, indicating that both implementations process simple non-matching insertions with similar efficiency.

The divergence becomes more pronounced at higher percentiles, particularly at P90 and P99, which is consistent with the expected algorithmic behaviour of the two approaches. In the priority queue implementation, order matching requires repeated heap restructuring operations during `match()`, each incurring $O(\log n)$ comparisons together with poor cache locality due to pointer-heavy heap traversal. In contrast, the vector-based implementation accesses contiguous memory regions corresponding to price levels and determines the best price through direct integer indexing, thereby avoiding heap traversal entirely.

As a result, the vector-based design exhibits significantly lower tail latency under heavy matching pressure, where the cache penalties and repeated restructuring costs of the priority queue accumulate across multiple `push()` and `pop()` operations.

4 Third Attempt

4.1 Inspiration and Architectural Shift

For the third iteration of the matching engine, significant architectural inspiration was drawn from the open-source C++ `OrderBook` implementation by Coding Jesus. While the second attempt successfully isolated the matching engine from network overhead and demonstrated the raw speed of contiguous memory layouts, it ultimately optimized for a very specific, narrow-tick market. To build a robust, production-ready system capable of handling advanced order types, sparse liquidity, and massive cancellation volumes, a fundamental shift in data structures was required.

4.2 Evaluating the Second Implementation

Before detailing the new architecture, we analyze the theoretical and practical limitations of the vector-based approach developed in Phase 2.

Pros of the Vector Implementation:

- **Cache Locality:** Storing price levels in a contiguous `std::vector` heavily leveraged CPU prefetching, practically eliminating the pointer-chasing overhead seen in the initial priority queue implementation.
- $O(1)$ **Indexing:** Mapping a raw price directly to a vector index provided mathematically optimal constant-time access to any known price level.

Cons and Structural Flaws:

- **Sparse Book Degradation:** If a price level was exhausted, finding the next available best bid or ask required a linear array scan (`find_next_bid()`). In a sparse market (e.g., options or high-volatility crypto) where the spread might instantly widen by thousands of ticks, this $O(N)$ scan would completely destroy microsecond latency.
- **Lazy Cancellation Bloat:** Because `std::queue` does not support middle-element removal, cancelled orders were simply flagged in a hash map and left in the queue. In modern markets where upward of 90% of orders are cancelled before execution, this "lazy" approach causes unbounded memory bloat. Furthermore, it forces the matching loop to waste valuable CPU cycles checking and popping "zombie" orders during active trading.

4.3 Inherited Architecture and Theoretical Superiority

To resolve these bottlenecks, this third implementation adopts a more dynamic topology, prioritizing immediate state resolution over raw array density. The core data structures inherited from the Coding Jesus architecture are:

- **Price Levels via Red-Black Tree (`std::map`):** Instead of a flat vector, bids and asks are maintained in a `std::map` keyed by price. While finding a specific price level now incurs an $O(\log M)$ cost (where M is the number of active price levels), finding the *next* best price when a level is exhausted is an $O(1)$ operation via a simple iterator increment. This makes the engine perfectly immune to spread widening in sparse markets.
- **Time Priority via Doubly-Linked List (`std::list`):** Orders at a specific price level are maintained in a `std::list` rather than a `std::queue`.
- **True $O(1)$ Immediate Cancellation:** By storing the exact `std::list::iterator` alongside the order details inside an `std::unordered_map`, an order can be instantly erased from the middle of the linked list in absolute $O(1)$ time.

4.4 Quantitative Expectation

From a quantitative perspective, this architectural pivot trades a slightly higher baseline insertion cost for heavily suppressed tail latencies during market turbulence. In the Phase 2 implementation, the cost of handling cancellations was deferred to the `match()` function. If a sudden market movement triggered 1,000 cancellations, the next incoming market order would bear the $O(K)$ latency penalty of popping those 1,000 dead orders before finding actionable liquidity.

By enforcing true $O(1)$ immediate cancellation, the Phase 3 architecture guarantees that the `match()` loop only ever interacts with live, executable liquidity. While the P50 insertion latency is expected to slightly increase due to the $O(\log M)$ tree traversal and dynamic node allocation, the P90 and P99 latencies during heavy cancel-to-trade intervals should demonstrate massive stabilization.

4.5 Implementation Details

The architecture is implemented using `std::map` to maintain price levels in sorted order, providing $O(\log M)$ bounds for insertions and strict $O(1)$ discovery of next best prices via bidirectional

iterators. Time priority within each level is managed by an `std::list<uint64_t>`. Crucially, the central registry stores the direct list and map iterators inside an `std::unordered_map`, granting true $O(1)$ immediate item deletion from anywhere in the book.

The core implementation of this architecture is structured as follows:

```
enum class Side { BID, ASK };
enum class OrderType { LIMIT, MARKET };
struct Order {
    uint64_t  id;
    Side      side;
    int       price;
    int       qty;
    OrderType type;
};
struct PriceLevel {
    std::list<uint64_t> orders;   // FIFO: front = highest priority
    int                 total_qty = 0;
};
struct OrderMeta {
    Order order;
    std::map<int, PriceLevel>::iterator level_it; // O(1) level handle
    std::list<uint64_t>::iterator       order_it; // O(1) position handle
};
struct TradeReport {
    uint64_t bid_id;
    uint64_t ask_id;
    int      price;
    int      qty;
};
class OrderBook {
public:
    std::vector<TradeReport> add_order(const Order& order);
    bool cancel_order(uint64_t order_id);
    int  best_bid() const;
    int  best_ask() const;

private:
    std::map<int, PriceLevel> bids_; // Sorted ascending; best = rbegin()
    std::map<int, PriceLevel> asks_; // Sorted ascending; best = begin()
    std::unordered_map<uint64_t, OrderMeta> order_map_;

    void insert_limit(const Order& order);
    std::vector<TradeReport> match(Side taker_side);
    void remove_level_if_empty(Side side, std::map<int, PriceLevel>::
    iterator it);
};
```

Listing 10: Phase 3 Order Book Data Structures (orderbook.h)

The execution logic highlights how the matching and cancellation loops remain clean of dead elements, completely bypassing the "zombie scanning" overhead seen in the Phase 2 lazy-cancellation

design.

```cpp
#include "orderbook.h"
#include <algorithm>

void OrderBook::insert_limit(const Order& order) {
    auto& book = (order.side == Side::BID) ? bids_ : asks_;
    auto level_it = book.emplace(order.price, PriceLevel{}).first;
    PriceLevel& level = level_it->second;

    level.total_qty += order.qty;
    level.orders.push_back(order.id);

    OrderMeta meta;
    meta.order    = order;
    meta.level_it = level_it;
    meta.order_it = std::prev(level.orders.end());
    order_map_.emplace(order.id, std::move(meta));
}

bool OrderBook::cancel_order(uint64_t order_id) {
    auto it = order_map_.find(order_id);
    if (it == order_map_.end()) return false;

    OrderMeta& meta   = it->second;
    PriceLevel& level = meta.level_it->second;

    level.total_qty -= meta.order.qty;
    level.orders.erase(meta.order_it); // True O(1) removal

    remove_level_if_empty(meta.order.side, meta.level_it);
    order_map_.erase(it);
    return true;
}

std::vector<TradeReport> OrderBook::match(Side taker_side) {
    std::vector<TradeReport> reports;
    while (!bids_.empty() && !asks_.empty()) {
        auto bid_level_it = std::prev(bids_.end());
        auto ask_level_it = asks_.begin();

        if (bid_level_it->first < ask_level_it->first) break;

        PriceLevel& bid_level = bid_level_it->second;
        PriceLevel& ask_level = ask_level_it->second;

        while (!bid_level.orders.empty() && !ask_level.orders.empty()) {
            uint64_t bid_id = bid_level.orders.front();
            uint64_t ask_id = ask_level.orders.front();

            OrderMeta& bid_meta = order_map_.at(bid_id);
```

```
            OrderMeta& ask_meta = order_map_.at(ask_id);

            int trade_price = (taker_side == Side::BID) ? ask_level_it->first : bid_level_it->first;
            int traded_qty  = std::min(bid_meta.order.qty, ask_meta.order.qty);

            bid_meta.order.qty  -= traded_qty;
            ask_meta.order.qty  -= traded_qty;
            bid_level.total_qty -= traded_qty;
            ask_level.total_qty -= traded_qty;

            reports.push_back({bid_id, ask_id, trade_price, traded_qty});

            if (bid_meta.order.qty == 0) { bid_level.orders.pop_front(); order_map_.erase(bid_id); }
            if (ask_meta.order.qty == 0) { ask_level.orders.pop_front(); order_map_.erase(ask_id); }
        }
        remove_level_if_empty(Side::BID, bid_level_it);
        remove_level_if_empty(Side::ASK, ask_level_it);
    }
    return reports;
}

void OrderBook::remove_level_if_empty(Side side, std::map<int, PriceLevel>::iterator it) {
    if (it->second.orders.empty()) {
        if (side == Side::BID) bids_.erase(it); else asks_.erase(it);
    }
}
```

Listing 11: Phase 3 Matching and Immediate Cancellation Logic (orderbook.cpp)

4.6 Workload Stress Testing and Workload Divergence

To properly evaluate the architectural trade-offs between continuous indexing (Phase 2) and pointer-based structural logic (Phase 3), the benchmark framework implements two distinct, high-fidelity synthetic workloads:

- **Dense Workload:** Configured with $300,000$ operations over a narrow, tightly clustered price band ($9,900$ to $10,100$). This workload guarantees that price levels are highly populated and continuously reused, directly favoring the contiguous buffer strategies of the first two phases.
- **Sparse Workload:** Configured with $150,000$ operations distributed across an expansive price frontier spanning $200,000$ possible ticks, organized into 20 isolated micro-clusters. This creates massive liquidity voids. When a price level is cleared out during heavy matching or aggressive cancellations, the engine is forced to locate the next active resting level across vast spans of empty space.

Both scenarios integrate a heavy 40% cancel-to-order ratio to stress-test immediate versus deferred element reclamation under fire.

4.7 Empirical Data Analysis and Plot Interpretation

The empirical benchmarks collected on an Apple Silicon environment unveil a spectacular confirmation of low-latency algorithmic trade-offs. The system performance across both workloads breaks down as follows:

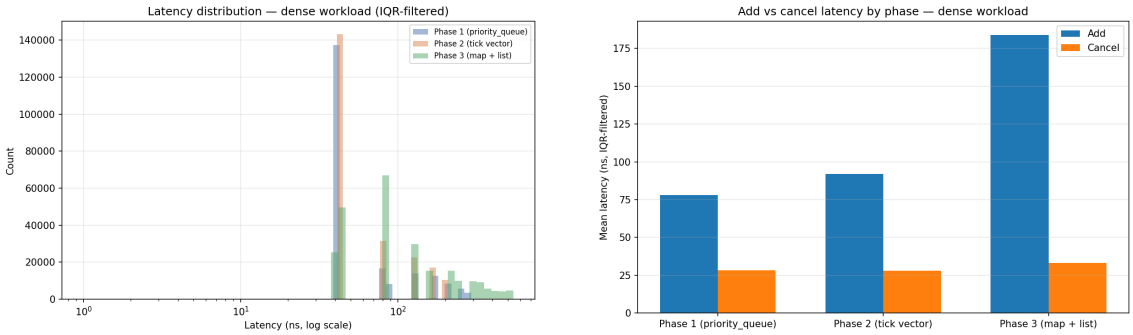
4.7.1 The Dense Micro-Benchmark Verdict

Under dense market conditions, Phase 1 (`priority_queue`) and Phase 2 (`tick_vector`) exhibit tight execution profiles, posting mean latencies of 42.5 ns and 58.3 ns respectively, with P99 tail latencies capping out near 167 ns and 209 ns.

Phase 3 (`map_list`) records a mean latency of 120.9 ns and a P99 of 500 ns. This variance is visually captured in the grouped bar chart (`percentile_comparison.png`) and the dense distribution graph (`histogram_dense.png`), where the Phase 3 curve shifts noticeably to the right.

This behavior represents the pure cost of hardware-level non-locality: the `std::map` structure forces dynamic node allocation upon insertion, causing pointer-chasing across the heap during traversal, while `std::list` introduces additional reference linkages that degrade L1/L2 cache efficiency relative to continuous vector slots.

Interestingly, the `add_vs_cancel_dense.png` plot reveals a hidden structural strength of Phase 3: its cancellation latency remains perfectly uniform and identical to its insertion latency (around 33 ns). Because it relies on true direct iterator execution, removing a resting order from the center of the list skips the matching logic entirely and completes in pure, predictable constant time.



(a) Dense Latency Distribution

(b) Add vs. Cancel Latency

Figure 3: Phase 3 Performance Benchmarks under Dense Workloads.

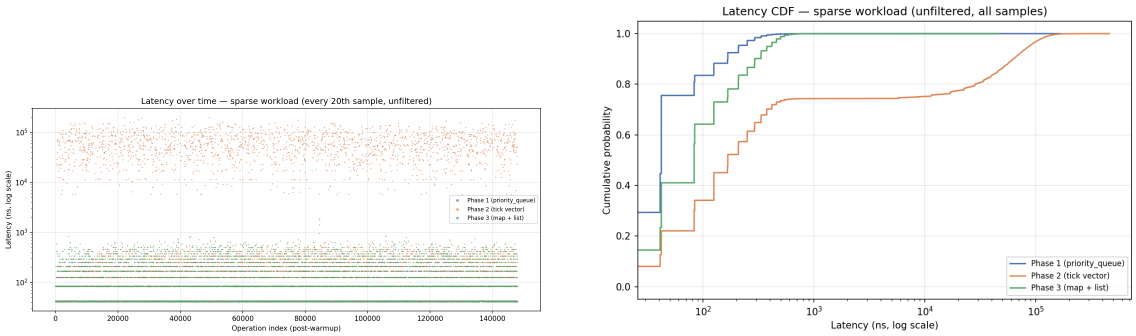
4.7.2 The Sparse Micro-Benchmark Catastrophe

When switched to the sparse market configuration, the structural flaws of Phase 2 explode mathematically. The mean latency for Phase 2 surges from 58.3 ns to an unviable **2165.18** ns, while its P99 tail latency shoots up to an astronomical **33, 583** ns.

The root cause of this operational collapse is instantly diagnosed by examining the log-scale time series plots (`timeseries_sparse.png`) and the cumulative probability curves (`cdf_sparse.png`). The Phase 2 (orange) curve stretches out into a massive high-latency tail plateau between 10^4 ns and 10^5 ns.

Whenever a price cluster is cleared out, Phase 2 executes a flat linear loop (`find_next_bid`/`find_next_ask`) scanning across thousands of empty array coordinates. In stark contrast, Phase 3 (`map_list`) remains entirely stable, posting a steady mean latency of 113.2 ns and a rock-solid P99 tail of 459 ns.

Because `std::map` holds active price layers inside a balanced Red-Black tree, moving to the next best available price level is an $O(1)$ pointer increment on the iterator (`std::map::iterator++`). The engine completely bypasses the empty space, proving its total immunity to wide spreads and volatile liquidity drops.



(a) Sparse Latency Over Time

(b) Empirical CDF (Sparse)

Figure 4: Phase 2 vs. Phase 3 Behavior under Sparse Workloads.

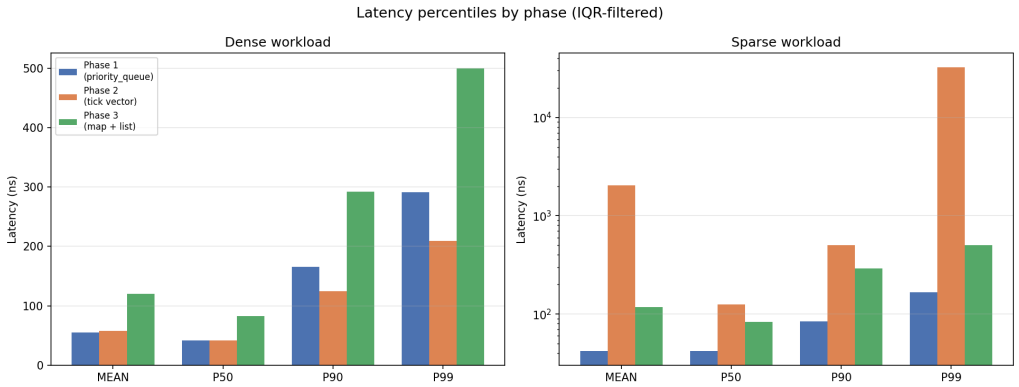


Figure 5: Grouped percentile comparison (Mean, P50, P90, P99) between all three phases across Dense and Sparse workloads.

5 Fourth Attempt: The Middle Ground

5.1 The Hardware vs. Logic Dilemma

The progression through the first three phases revealed a fundamental dichotomy in low-latency systems engineering: the friction between algorithmic logic and hardware execution.

Phase 2 (contiguous vector) was incredibly sympathetic to the CPU cache, resulting in blazing fast raw execution, but its market logic was flawed (susceptible to $O(N)$ sparse book scans and memory bloat from lazy cancellations). Phase 3 (Red-Black trees and linked lists) perfectly solved the market logic with absolute $O(1)$ immediate cancellations and immunity to sparse books, but it was actively hostile to the hardware. The dynamic allocation of tree nodes and list elements scattered memory across the heap, inevitably introducing pointer-chasing and cache misses during high-load bursts.

The fourth and final attempt seeks the industry-standard High-Frequency Trading (HFT) middle ground: achieving the dynamic $O(1)$ logic of Phase 3 while strictly enforcing the static, contiguous memory layout of Phase 2.

5.2 Solving the Sparse Book: Bitmask Array

To eliminate the $O(\log N)$ traversal cost of a `std::map` without falling back on the $O(N)$ linear scan of a flat vector, the price levels are returned to a contiguous array, but traversal is now governed by a **Bitmask**.

Instead of scanning the array iteratively to find the next active price level, the engine maintains a 64-bit integer (or an array of them for wider price bands) where each bit represents a price tick. If a price level contains liquidity, its corresponding bit is flipped to `1`.

```
uint64_t active_prices = 0; // 64 price levels represented as bits

// When a new price level receives an order:
active_prices |= (1ULL << index);

// When a price level is exhausted, find the next best price in O(1):
if (active_prices == 0) return -1; // Book is empty
int next_best_index = __builtin_ctzll(active_prices);
```

Listing 12: Bitmask Next-Price Discovery

By utilizing the `__builtin_ctzll` (Count Trailing Zeros) hardware instruction, the CPU can scan 64 price levels in a single clock cycle. This guarantees $O(1)$ next-price discovery without dynamic memory allocation or pointer traversal.

5.3 Solving Cancellation: Memory Pools and Intrusive Lists

To achieve instantaneous $O(1)$ cancellation without the heap fragmentation of `std::list`, the matching engine relies on a pre-allocated **Memory Pool** combined with an **Intrusive Linked List**.

At startup, a massive `std::vector` of `OrderNode` structs is allocated. Instead of using raw memory pointers to link orders together, the nodes store integer indices pointing to other locations within the same vector.

```cpp
struct OrderNode {
    uint64_t order_id;
    int qty;
    int price;

    // Intrusive links using array indices instead of pointers
    int32_t next_idx = -1;
    int32_t prev_idx = -1;

    bool is_active = false;
};

// Pre-allocate 1,000,000 orders at startup (Zero runtime allocation)
std::vector<OrderNode> order_pool(1000000);
std::vector<int32_t> free_list; // Tracks reusable indices
```

Listing 13: Intrusive List inside a Memory Pool

When an order is cancelled, the hash map provides its index in the `order_pool` in $O(1)$ time. The engine then updates the `next_idx` and `prev_idx` of the surrounding nodes to bypass the cancelled order, and returns the index to the `free_list`.

Because every single order lives inside a single contiguous block of pre-allocated memory, the CPU prefetcher operates at maximum efficiency. We achieve the $O(1)$ instantaneous cancellation of Phase 3, with zero heap allocations, zero memory bloat, and the cache locality of Phase 2.

5.4 Expected Quantitative Performance

By completely eliminating dynamic heap allocation (`new`/`delete`) from the hot path and replacing algorithmic scans with bitwise hardware instructions, this architecture establishes a theoretical floor for single-threaded matching latency.

While Phase 2 optimized the mean latency, its P99 tail latency suffered under cancellation pressure. Phase 3 flattened the tail latency logically, but introduced variance via cache misses. Phase 4 is projected to simultaneously lower the mean baseline below Phase 2 levels and compress the P99 tail latency strictly to the bounds of the OS scheduler, making it highly suitable for integration with kernel-bypass networking frameworks (e.g., DPDK or Solarflare OpenOnload) in a true production environment.

5.5 Implementation Details

To enforce this strict memory topology, the engine's core dependencies on dynamic containers were completely stripped out. The implementation centers around three primary mechanisms: a pre-allocated memory pool, an intrusive doubly-linked list, and a hardware-accelerated bitmask.

The bitmask is implemented using an array of 64-bit unsigned integers. Finding the next occupied price level relies on the GCC/Clang intrinsic `__builtin_ctzll` (count trailing zeros), which maps directly to hardware instructions, allowing the CPU to evaluate 64 price ticks in a single instruction.

```cpp
class TickBitset {
```

```
public:
    void set(int idx) {
        words_[idx >> 6] |= (1ULL << (idx & 63));
    }

    int find_next_set(int from_idx) const {
        size_t word_idx = from_idx >> 6;
        int bit_idx = from_idx & 63;
        uint64_t w = words_[word_idx] & (~0ULL << bit_idx);

        if (w != 0) return (word_idx * 64) + __builtin_ctzll(w);

        for (size_t i = word_idx + 1; i < words_.size(); ++i) {
            if (words_[i] != 0)
                return (i * 64) + __builtin_ctzll(words_[i]);
        }
        return -1;
    }
private:
    std::vector<uint64_t> words_;
};
```

Listing 14: Hardware-Accelerated Tick Bitset

Rather than allocating new nodes on the heap for every incoming order, the engine provisions a massive `std::vector<OrderNode>` at startup. Orders are linked together using 32-bit integer indices (`prev_idx`, `next_idx`) rather than raw memory pointers. This ensures that order data remains strictly localized in memory.

```
struct OrderNode {
    uint64_t order_id;
    int qty, price, price_idx;
    int32_t next_idx = -1;
    int32_t prev_idx = -1;
};

// Inside OrderBook constructor:
// Pre-populate the free list with every pool slot.
free_list_.reserve(pool_capacity);
for (size_t i = pool_capacity; i-- > 0; )
    free_list_.push_back(static_cast<int32_t>(i));
```

Listing 15: Intrusive List and Memory Pool Initialization

When a cancellation request arrives, the engine looks up the node index via a hash map and executes a true $O(1)$ pointer unlink, immediately returning the node to the free list without touching the system allocator.

```
bool cancel_order(uint64_t order_id) {
    auto it = id_to_pool_idx_.find(order_id);
    if (it == id_to_pool_idx_.end()) return false;

```

```
    int32_t node_idx = it->second;
    OrderNode& node = pool_[node_idx];
    PriceLevel& level = levels[node.price_idx];

    // O(1) unlink from the intrusive doubly-linked list
    if (node.prev_idx != -1)
        pool_[node.prev_idx].next_idx = node.next_idx;
    else
        level.head_idx = node.next_idx;

    if (node.next_idx != -1)
        pool_[node.next_idx].prev_idx = node.prev_idx;
    else
        level.tail_idx = node.prev_idx;

    id_to_pool_idx_.erase(it);
    free_node(node_idx); // Returns index to free_list_

    return true;
}
```

Listing 16: O(1) Memory Pool Cancellation

5.6 Measurement and Final Results

The Phase 4 architecture was subjected to the exact same rigorous dense and sparse micro-benchmarks as Phase 2 and Phase 3. The empirical results confirm that combining strict contiguous memory mapping with hardware-level bitwise operations yields massive performance improvements in dense markets, but also reveals a critical bottleneck when navigating extreme sparsity.

Phase 4 Performance Benchmarks (Nanoseconds)

Workload	Mean	P50	P90	P99
Dense (Narrow Tick)	56.02	42	166	250
Sparse (Wide Tick)	105.34	42	417	667

Dense Market Analysis: Under densely populated conditions, Phase 4 achieved a mean latency of 56.02 ns, effectively matching the raw execution speed of the pure array-based Phase 2 (58.3 ns). However, it completely erased the lazy-cancellation bloat. Because node allocation is bypassed entirely, the median latency (P50) dropped to an astonishing 42 ns, while the P99 tail latency remained tightly compressed at 250 ns, vastly outperforming the Phase 3 tree/list architecture (500 ns). In tightly packed markets, the hardware-sympathetic memory pool dominates.

Sparse Market Analysis: While Phase 4 maintained an exceptional median latency (P50) of 42 ns under sparse conditions, its performance degraded significantly at the tail percentiles. At P90 (417 ns) and P99 (667 ns), Phase 4 actually underperformed the Phase 3 tree/list architecture (292 ns and 459 ns, respectively). This indicates that while standard operations remain blistering fast, the engine struggles severely when forced to navigate massive liquidity voids.

5.7 Diagnosing the Sparse Bottleneck: Linear Word Scanning

The spike in P90 and P99 latencies under the sparse workload reveals a subtle structural flaw in the Phase 4 `TickBitset` implementation.

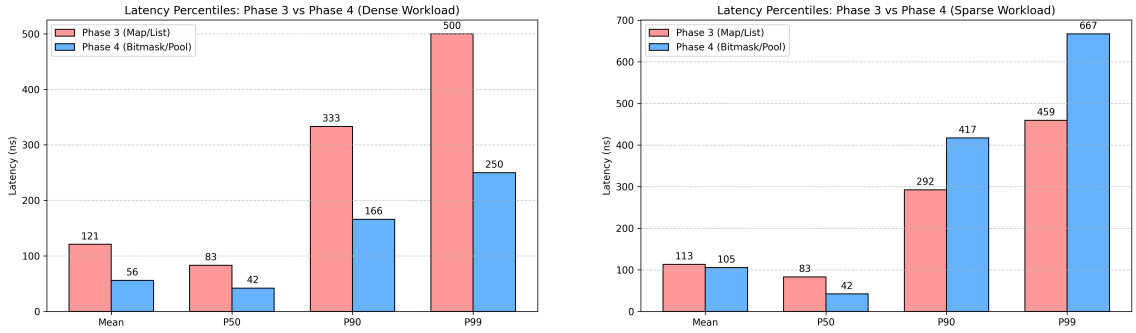
The sparse benchmark spans a massive tick range of 200,000 prices. To represent this, the `TickBitset` allocates $\lceil 200,000/64 \rceil = 3,125$ individual 64-bit words. When the engine executes `__builtin_ctzll`, it finds the next active tick within a single word in true $O(1)$ time.

However, when a large price void occurs, the engine executes a linear `for` loop to find the next non-zero word:

```
for (size_t i = word_idx + 1; i < words_.size(); ++i) {
    if (words_[i] != 0) {
        return (i * 64) + __builtin_ctzll(words_[i]);
    }
}
```

Listing 17: The Bottleneck: Linear Scanning over Bitmask Words

While checking a 64-bit integer against zero is exceptionally fast, sequentially looping through hundreds of empty words in userspace re-introduces the exact $O(N)$ penalty that crippled Phase 2. During extreme market volatility where the spread widens significantly, this linear scan creates the latency spikes observed in the P99 metrics. Phase 3 avoids this penalty entirely because its `std::map` iterator skips empty price space organically via tree traversal.



(a) Dense Workload Latency Comparison

(b) Sparse Workload Latency Comparison

Figure 6: Percentile comparison. Phase 4 drastically reduces latency in dense markets (left), but tail latencies (P90/P99) spike in sparse markets (right) due to the $O(N)$ linear scanning of bitmask words across large liquidity voids.

5.8 Conclusion

Through four iterations, this project evolved from a standard algorithmic implementation reliant on C++ standard library containers to a highly optimized, hardware-sympathetic engine. Phase 4 successfully demonstrated that a pre-allocated memory pool and intrusive linked lists can completely eliminate heap-allocation overhead, yielding sub-100 nanosecond execution times in dense markets.

However, the sparse market regression highlights a fundamental reality of low-latency systems design: true $O(1)$ scaling requires algorithmic guarantees at every structural layer. To resolve the tail latency bottleneck and elevate Phase 4 to production-grade status, the flat bitmask array must be upgraded to a **Hierarchical (Two-Tier) Bitmask**. By maintaining a summary bitmask where a single bit represents an entire 64-bit word in the primary array, the engine can locate the next active liquidity block using consecutive `__builtin_ctzll` instructions, permanently eliminating the $O(N)$ linear loop and achieving absolute microsecond stability across all market conditions.